CSCI-6461 Computer System Architecture

PROJECT - Part Two – Simulator

Designed by TEAM 6

Owen Chibanda

Deepika Kubendira Rao

Nongxu Li

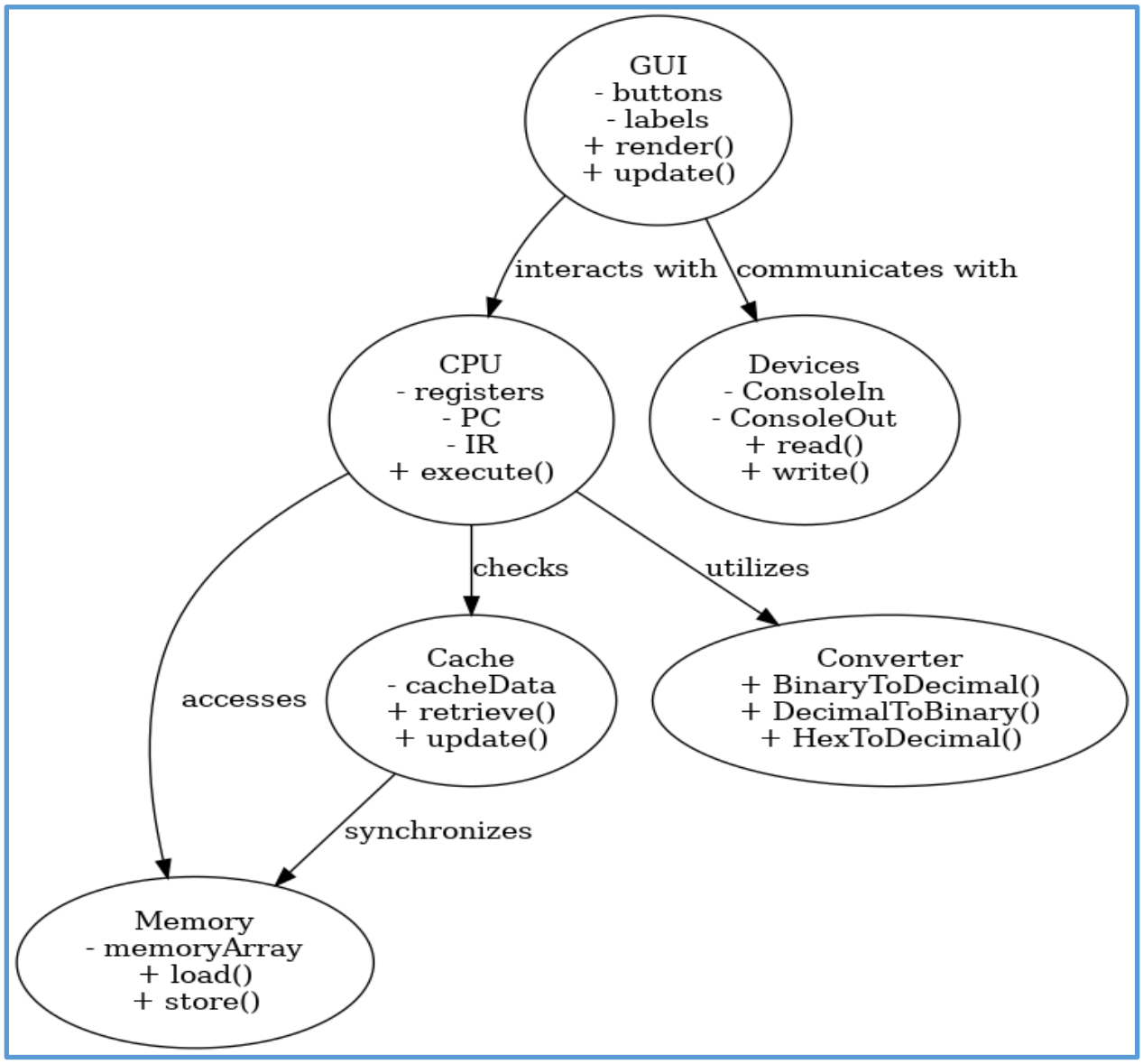
GITHUB LINK:
https://github.com/deepikarao281199/CSCI_6461_PROJECT_TEAM6/tree/Simulator-Project-Part-II

1. Project Overview

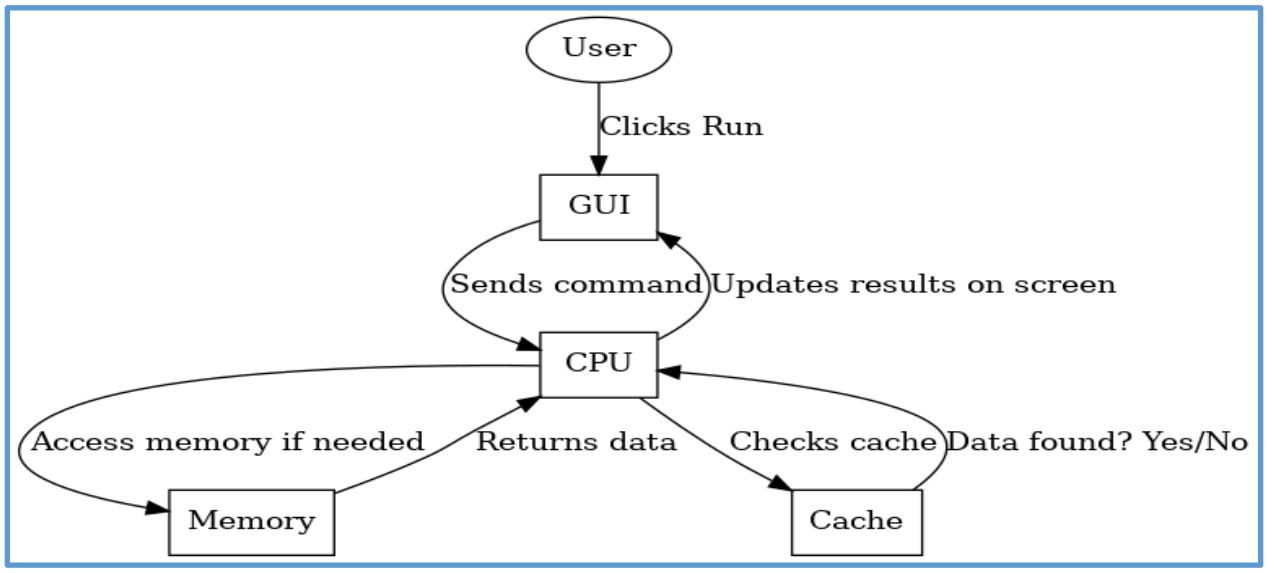
The CSA Simulator is a software tool that replicates key functions of a computer system, including the CPU, memory, I/O devices, and data conversion mechanisms. Built with Java, the simulator provides an interactive graphical user interface (GUI) using Swing, allowing users to control CPU processes, memory management, and device interaction.

UML Diagrams

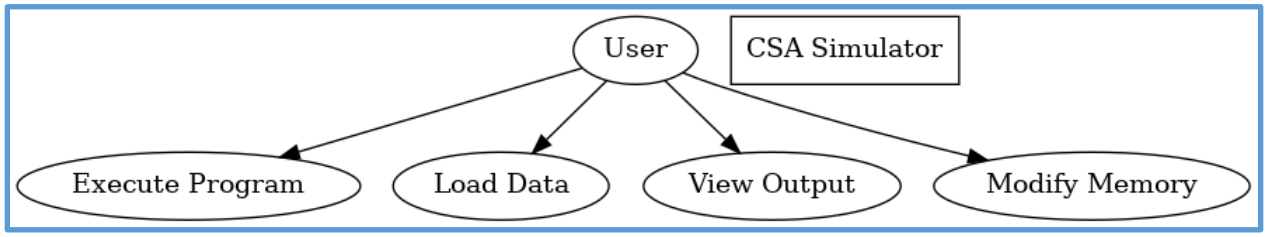
UML Class Diagram

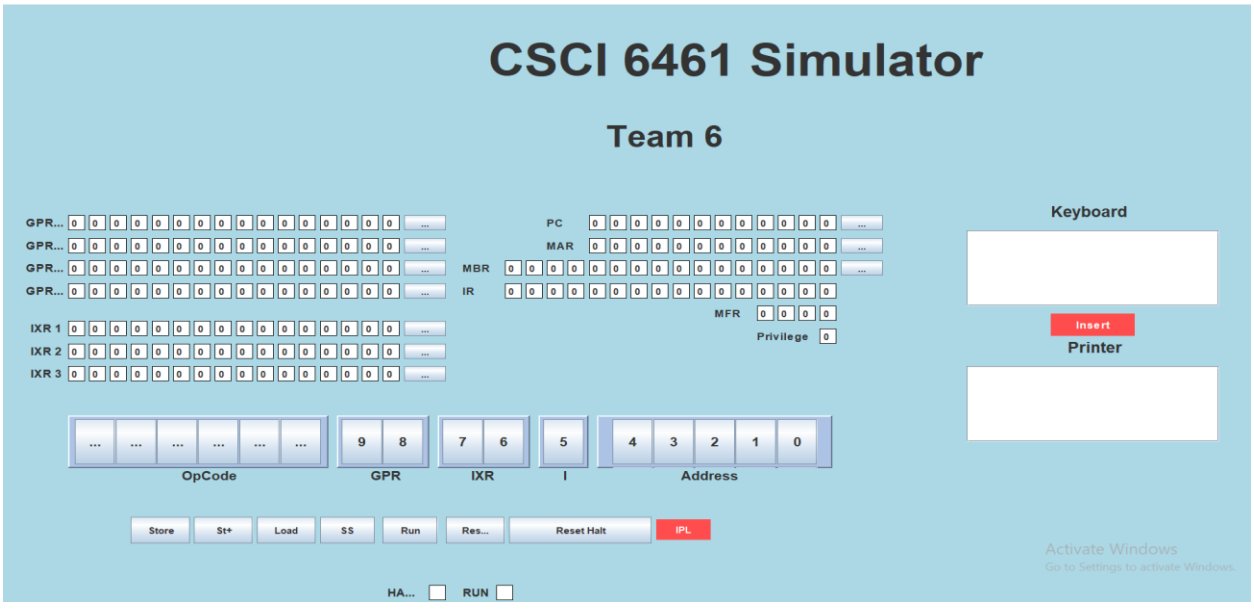


UML Sequence Diagram



UML use-case Diagram





2. System Components

Memory Management (Memory.java)

- **Memory Array:** A 2KB memory system is simulated using a short[] array of 2048 cells, all initialized to zero.
- **Fixed Memory Configuration:** The memory size is predetermined, but future implementations could introduce dynamic allocation for greater flexibility.

Graphical User Interface

- **Java Swing-Based UI:** The interface is constructed using Java Swing components, such as JLabels and JButtons, to visually represent registers, memory structures, and control buttons.
- **Core GUI Elements:**
    - **Registers and Labels:** Provides a visual representation of General Purpose Registers (GPRs), Index Registers (IXRs), and the Machine Fault Register (MFR).
    - **Control Buttons:** Includes "Store," "Load," "Reset," and "Run" for user interaction.
    - **Event Handling:** Each button is linked to corresponding methods like Store(), LoadValue(), and RunProg(), ensuring appropriate function execution.
    - **Threading for Efficiency:** To maintain GUI responsiveness, the "Run" function executes on a background SwingWorker thread.
    - **Static Layout Design:** The reliance on setBounds() results in a fixed layout, potentially limiting adaptability to different screen resolutions.

Devices Interface (Devices.java)

- **Console I/O Simulation:**
    - **Console Output:** Displays output using a monospaced font and a green-on-black color scheme.
    - **Console Input:** Mimics keyboard interactions through a JTextArea.
- **Panel-Based Design:** Components are grouped into panels with labeled borders for better organization.
- **Manual Layout Constraints:** The reliance on setBounds() restricts the GUI's responsiveness to window resizing.

Number Conversion Utility (Converter.java)

- **Data Conversion Methods:** Supports conversions among binary, decimal, and hexadecimal values.
    - BinaryToDecimal(): Converts binary values to decimal format.
    - DecimalToBinary(): Converts decimal numbers to binary representation.
    - HexToDecimal(): Transforms hexadecimal strings into decimal values.
- **Code Reusability:** The modularity of conversion functions allows seamless integration across different simulator components.

CPU Simulation (CPU.java)

- **Register Management:** Manages GPRs, Program Counter (PC), Instruction Register (IR), and additional control registers.
- **Instruction Execution:** The Execute(Memory) function deciphers binary opcodes and executes corresponding operations such as loading, storing, and halting.
- **Opcode Handling:** Implements a switch-case structure to support instructions like LDR (Load Register), STR (Store Register), and HLT (Halt).
- **Memory Fault Detection:** The Memory Fault Register (MFR) tracks and addresses memory access violations.
- **Inheritance Model:** The CPU class extends Converter.java, leveraging conversion utilities for opcode interpretation and memory addressing.

Cache Management (Cache.java)

- **Performance Enhancement:** The newly integrated cache mechanism stores frequently accessed data, improving retrieval speed.
- **Optimized Data Retrieval:** The CPU first queries the cache before accessing main memory, reducing processing delays.
- **Synchronization:** Ensures consistency between cached data and main memory.

Project Structure

- **src/main/java:** Core project source code.
    - **com.csa.simulator.components:** Houses major system components.
        - **Assembler:** Handles program parsing and instruction loading.
        - **Cache:** Manages frequently accessed data.

        - **CacheData:** Implements cache data processing.
        - **Transformer:** Facilitates conversions between binary, decimal, and hexadecimal.
        - **CPU:** Executes instructions and manages registers.
        - **Modules:** Manages I/O device interactions.
        - **Memory:** Implements memory storage and retrieval operations.
- **com.csa.simulator.gui:** Manages user interface components.
    - **GUI:** Primary UI class.
    - **Main:** Application entry point.
- **src/main/resources:** Holds additional project resources.
    - **Program1.txt:** Sample program for execution testing.
- **src/test:** Reserved for unit testing.
- **target:** Contains Maven-generated artifacts.
- **pom.xml:** Maven configuration file for dependency and build management.
- **README.md:** Documentation outlining setup instructions and project details.
- **.gitignore:** Defines files to be excluded from version control.

4. Key Enhancements in the Simulator

Cache Implementation

- **Newly Added Cache System:** Enhances processing efficiency by reducing direct memory access delays.
- **Cache Synchronization:** Ensures coherence between cache-stored data and main memory.

Expanded Instruction Set

- **Additional Instructions:** Introduces new arithmetic, logical, and control operations, broadening the simulator's capabilities.
- **Comprehensive Instruction Support:** Enables execution of the full instruction set defined in the project specifications.

Program 1 Execution Capability

- **Program1.txt Integration:** The simulator now supports loading and executing test programs.
- **Execution Workflow:** Updates to the Assembler and CPU classes allow seamless interpretation and execution of instructions.
- **User Input Functionality:** Accepts 20 user-input values and determines the closest match from the dataset.

5. Technical and Design Considerations

- **GUI Layout Management:** The current reliance on setBounds () limits flexibility; adopting layout managers such as GridLayout or BorderLayout would enhance adaptability.
- **Threading for Performance:** Background processing through SwingWorker ensures uninterrupted GUI responsiveness.
- **Memory Expansion Potential:** The 2KB memory limitation could be addressed by implementing a scalable allocation strategy.
- **Utility Code Reusability:** The decoupled structure of Converter.java promotes efficiency and modularity across various system components.